Decision rules: serving an LLM larger than free memory, from one NVMe drive

For platform engineers and inference operators. One page; the evidence is in `REPORT.md`.

Measured on one node: AMD Ryzen AI Max+ 395, 125 GB unified RAM, one NVMe at 7.1 GB/s, k3d, PyTorch on CPU, bf16 weights, no quantisation. Six models, 1,431 runs.

Step 1. Compute your floor

```
floor per token = bytes you must re-read per token ÷ saturated disk bandwidth
```

Measure the bandwidth with four parallel readers; more readers do not add speed. Plain layer streaming matched this floor within 5% on all six models tested (0.37 s for a 1.5B dense model up to 8.4 s for a 30B mixture of experts on a 7.1 GB/s drive).

Step 2. Pick the approach by architecture

Your model	Do this	Expect (per token, vs fully resident)	Memory saved
Dense	One pod streams the layers with 4 read slots; add tree speculation with a small drafter from the same family	1.5–2.9× (3B–14B)	51–74%
Mixture of experts	Memory-map the weights and set `resources.limits.memory` on the pod; warm it up before traffic	1.7–5.2× (Qwen3-30B-A3B), 3.0–8.0× (gpt-oss-20b), depending on the limit	29–73%
Many concurrent users	Add batching on top	1.19× per user at 512 users (dense 3B)	same

Step 3. Avoid these

Don't	Why	Measured
Kernel paging on a dense model	Every token touches every weight, so every token re-faults the model	8.5–14.9×
Streaming whole layers of a mixture of experts	Reads all experts, a token uses 6–12% of them	31.8–37.2×
Spreading layers over several pods to hide loads	Each pod is one more reader on the same drive; one pod with 4 slots reaches the same floor with less RAM	781 ms (4 pods, 3.25 GB) vs 798 ms (1 pod, 2.14 GB)
Pinning most layers and streaming a few	Costs before it saves anything	1.33× (dense), 3.9× (experts) at 0% saved
A drafter that is large next to the target	Its memory comes off the saving	1 GB drafter on a 3.1 GB model: only 24–28% saved
Trusting a cache benchmark with a repetitive prompt	Repeated tokens keep the same experts hot	1.87× measured, 7.4–8.0× real

Kubernetes recipes

Example manifests, generated by `src/k8s/deploy.py` (the same template used for every measured run), are in `src/k8s/examples/`:

Manifest	Approach
`02-dense-single-pod-streaming.yaml`	Dense: one pod, 4 read slots
`03-dense-streaming-tree-speculation.yaml`	Dense: one pod, 4 slots, 64-node tree with a 0.5B drafter
`04-moe-kernel-paging-38Gi.yaml`	Mixture of experts: memory-mapped weights, 38 GiB pod memory limit
`01-baseline-resident.yaml`	Reference: whole model resident
`05-cyclic-4-pods-original-idea.yaml`	The multi-pod scheme that the rules advise against, for comparison

For kernel paging the limit only reduces host memory if no other process keeps the weight files cached: drop the page cache for those files after the pod has started.